

Laboratorio di Architetture Software e Sicurezza Informatica [AAF1569]

6 - Rails (part 2)



SAPIENZA
UNIVERSITÀ DI ROMA

Part of therein contents are based on slides of Prof. Armando Fox

DIAG

Dipartimento di Ingegneria
informatica, automatica e gestionale
Antonio Ruberti

More on Routes

In the last lecture, we discussed how the Rails routing subsystem prepares a hash called `params[]` that is made available to the controller

Command `rake routes` will show the routes implied from file `routes.rb`

For example, the route for `show` will appear as `GET movies/:id` where `params[:id]` will hold the actual ID value parsed from the URI

Rails also provides an easy way to generate HTML forms in which form fields are named in such a way that another value in `params[]` will ease object creation

Default route URIs match the resource name, but we can override this behavior using **route helpers**: those decouple what the route does from its actual URI.

Routes and Controllers

Controller methods (**actions** in Rails) handle RESTful operations and also follow conventions. For example, let us consider a `routes.rb` file containing a single line `resources 'movies'` - this will result in the following REST routes:

Helper method	URI returned	RESTful route and action	
<code>movies_path</code>	<code>/movies</code>	GET <code>/movies</code>	index
<code>movies_path</code>	<code>/movies</code>	POST <code>/movies</code>	create
<code>new_movie_path</code>	<code>/movies/new</code>	GET <code>/movies/new</code>	new
<code>edit_movie_path(m)</code>	<code>/movies/1/edit</code>	GET <code>/movies/:id/edit</code>	edit
<code>movie_path(m)</code>	<code>/movies/1</code>	GET <code>/movies/:id</code>	show
<code>movie_path(m)</code>	<code>/movies/1</code>	PUT <code>/movies/:id</code>	update
<code>movie_path(m)</code>	<code>/movies/1</code>	DELETE <code>/movies/:id</code>	destroy

How should the controller class look like?

Routes and Controllers

The default routes will expect a file `controllers/movies_controller.rb` that defines a class `MoviesController` (similarly to `ActiveRecord::Base`, the class will descend from `ApplicationController`) that defines **7 instance methods** (`index, ...`) corresponding to the RESTful actions from the table.

Such controller actions will typically follow this pattern:

1. Collect the RESTful **request information** (parameters, resource ID, etc.)
2. Determine what **ActiveRecord operations** are necessary to fulfill the request (e.g., the Index action may require retrieving all movies from table Movies)
3. Set **instance variables** for what needs to be displayed in the view
4. **Render a view** to be returned as result for the request

Controllers and Views

How does each controller action select a view and pass information to it?

When a controller action ends, Rails will identify and render a view named:

`app/views/<model-name>/<action>.html.erb`

ActionView::Base handles views. The view here consists of HTML interspersed with Embedded Ruby (erb) tags that allow the results from running Ruby code to be interpolated in the HTML view. For example, we can access instance variables.

But how instance variables from one class (`MoviesController` in the example) become available to an object from another class (`ActionView::Base`)? Rails uses Ruby's *metaprogramming* to copy controller instance variables into it :-)

HTML

The HyperText Markup Language (**HTML**) is the **standard markup language for documents** designed to be **displayed in a web browser**.

Web browsers receive HTML documents from a web server and render them.

HTML describes the **structure of a web page** and comes with (some) cues for its appearance. It can be assisted by technologies such as CSS and scripting languages such as JavaScript.

Cascading Style Sheets (CSS) is a style sheet language for describing how to present a document written in a markup language like HTML. CSS is designed to enable the separation of presentation and content (e.g., layout, colors, and fonts)

HTML

```
<!DOCTYPE html>
<html>
  <head>
    <title>This is a title</title>
  </head>
  <body>
    <div>
      <p>Hello world!</p>
    </div>
  </body>
</html>
```

Text in `<html> .. </html>` describes the web page

Text in `<body> .. </body>` is the visible content

Tag `<div>` defines a division of the page and can be used for easy styling with CSS

A `<meta>` element placed in `<head> .. </head>` can be used to define a piece of page metadata.

The **general form** of an HTML element is:

```
<tag attribute1="value1"
attribute2="value2">content</tag>
```

The views we saw aren't legal HTML: they lack DOCTYPE, `<html>`, `<head>`, and `<body>`. These tags can be found instead in the special file `views/application.html.erb` that wraps all views by default.

Collecting Data from Users

How do we display a fill-in form to the user? Rails uses an **action** (and its route)

Movie creation in RottenPotatoes:

- The route **GET /movies/new** names the action, and the route helper **new_movie_path** will generate the URI portion of the route
- The action will be handled by the method **MoviesController#new**
- The controller action will render a view **app/views/movies/new.html.erb**

Intuitively, the view should contain an HTML form. The controller action must have knowledge about how the HTML form fields are named, so it can extract data and populate a Movie object. For this, Rails uses **form tag helper methods**.

Form Tag Helper

 `form_helper_example.html.erb`

```
1 <h2>Create New Movie</h2>
2 <%= form_tag movies_path, :method => :post, :class => 'form' do %>
3   <%= label :movie, :title, 'Title', :class => 'form-control' %>
4   <%= text_field :movie, :title, :class => 'form-control' %>
5   <%= label :movie, :rating, 'Rating', :class => 'form-control' %>
6   <%= select :movie, :rating, ['G','PG','PG-13','R','NC-17'], :class => 'form-control' %>
7   <%= label :movie, :release_date, 'Released On' %>
8   <%= date_select :movie, :release_date, :class => 'form-control' %>
9   <%= submit_tag 'Save Changes' %>
10 <% end %>
```

Arguments: URI where to submit and hash of optional arguments (such as the HTTP method to use for submitting the form)

HTML Forms

The `<form>` HTML element represents a document section containing interactive controls for submitting information.

https://guides.rubyonrails.org/form_helpers.html

<https://developer.mozilla.org/en-US/docs/Web/HTML/Element/form>

<https://developer.mozilla.org/en-US/docs/Web/HTML/Element/input>

Submitted form data is put into the **params** hash

First name:

Last name:

Acting on Collected Data

How is the info filled in by the user made available to the controller action?

Movie creation in RottenPotatoes:

- As of now, we arranged to have the form submitted to the create controller method. We need to have the **create** (or **update**) **controller action** parse the form information and use it to create a new movie in the database.
- The form fields created by the form tag helper have names of the form `'movie[title]'` and so on - **fields form a hash** supplied through `params`
- We can pass it along directly: `Movie.create!(params['movie'])`

Now, we only need to notify the user that the action has been completed.

DRY Redirection and flash[]

To notify of completion or failure, having a separate view may be clumsy. The DRY way would be issuing HTTP redirect to an already existing view (which can be configured, for example, to show a message at the top when applicable)

`flash[]` is a special object available in a controller, often used in conjunction with a `redirect_to` Rails method. If we put something into it during the current controller action, we can access it during the very next action (and only in it).

`flash[:notice]` for informational messages; `flash[:alert]` for errors

Where? `apps/views/layouts/application.html.erb` is a good candidate

DRY Ways to Write MVC Code

We will learn three mechanisms that Rails provide to reduce MVC code writing

1. **View partials** (to express reusable chunks of a view)
2. **Model validations** (check data validity for model manipulations)
3. **Controller filters** (check or set up common conditions for actions)

View Partials

In Rails, a **partial** is piece of content that can appear in multiple views.

Its name must begin with an *underscore*, to not be used in code that references it

A partial has also access to the instance variables of the views that includes it.

```
<!-- ...other code from index.html.erb here... -->
<div class="row bg-dark text-white">
  <div class="col-6 text-center">Title and More Info</div>
  <div class="col-2 text-center">Rating</div>
  <div class="col-4 text-center">Release Date</div>
</div>
<%= render partial: 'movie', collection: @movies %>
```



```
<div class="row">
  <div class="col-8"> <%= link_to movie.title, movie_path(movie) %> </div>
  <div class="col-2"> <%= movie.rating %> </div>
  <div class="col-2"> <%= movie.release_date.strftime('%F') %> </div>
</div>
```

Model Validation

Validation checks are triggered by calls to the method `valid?` done either directly in the code or indirectly when you try to **save the model** to the database

https://guides.rubyonrails.org/v6.1/active_record_validations.html

Validation errors are recorded in an `ActiveModel::Errors` object associated with each model; this object is returned by the instance method `errors`

Validation can be conditional (i.e., applied only under user-specified conditions)

Validations are a special case of Active Record lifecycle callbacks, which allow you to intercept a model object at various relevant points of its lifecycle

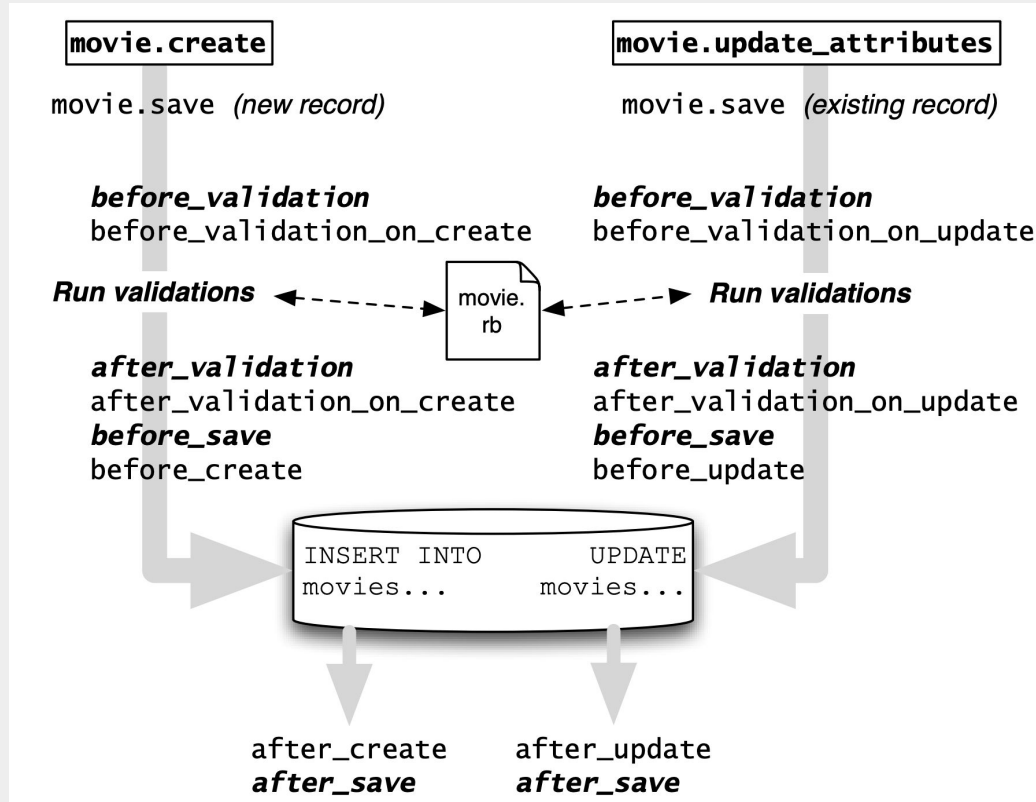
Model Validation

 validation_example.rb

```
1 class Movie < ActiveRecord::Base
2   def self.all_ratings ; %w[G PG PG-13 R NC-17] ; end # shortcut: array of strings
3   validates :title, :presence => true
4   validates :release_date, :presence => true
5   validate :released_1930_or_later # uses custom validator below
6   validates :rating, :inclusion => {:in => Movie.all_ratings},
7     :unless => :grandfathered?
8   def released_1930_or_later
9     errors.add(:release_date, 'must be 1930 or later') if
10       release_date && release_date < Date.parse('1 Jan 1930')
11   end
12   @@grandfathered_date = Date.parse('1 Nov 1968')
13   def grandfathered?
14     release_date && release_date < @@grandfathered_date
15   end
16 end
```

Console interactivity: <https://gist.github.com/armandofox/89adee067f097c7167e38e2c40a555aa>

Model Validation



Model Validation

before_save_example.rb

```
1 class Movie < ActiveRecord::Base
2   before_save :capitalize_title
3   def capitalize_title
4     self.title = self.title.split(/\s+/).map(&:downcase).
5       map(&:capitalize).join(' ')
6   end
7 end
```

This `before_save` callback will capitalize each word of a movie title, downcase the rest of the word, and compress multiple spaces between words to a single space.

Controller Filters

A **controller filter** is a method that checks whether certain conditions are true before an action is run, or sets up common conditions that many actions rely on

If the conditions are not fulfilled, the filter can choose to “stop the show” by rendering a view template or redirecting to another action

https://guides.rubyonrails.org/v6.1/action_controller_overview.html#filters

Filters normally apply to all actions in the controller, but their use can be confined to certain actions using `:only` or `:except` appropriately

Filters are run in the order they are declared

Controller Filters

 application_controller.rb

```
1 class ApplicationController < ActionController::Base
2   before_filter :set_current_user
3   protected # prevents method from being invoked by a route
4   def set_current_user
5     # we exploit the fact that the below query may return nil
6     @current_user ||= Moviegoer.where(:id => session[:user_id])
7     redirect_to login_path and return unless @current_user
8   end
9 end
```

Typical use case: remembering a user after authentication. As we will see later in the course, Rails controllers have a `session[]` for each user where you can store small data amounts that will be persisted between requests. Here, we validate by `user_id`.

Controller Filters vs. Model Validation

	Validation	Filter
Advice (DRYness)	Check invariants on model	Check conditions for allowing controller action to run
<i>Pointcut</i>	AR model lifecycle hooks	Before and/or after any public controller method
Can change execution flow?	No	Yes
Can define <i>advice</i> in arbitrary function?	Yes; shortcuts provided for common cases	Yes, must provide function
Info about errors?	Each model object has associated <i>errors</i> object	Capture in <i>flash[]</i> , <i>session[]</i> , or instance variable

Debugging

With so many moving parts, things might inevitably go wrong.

Use good tools: IDEs for syntax errors, work with your colleagues

Obviously, tools are not enough. You can resort to these activities:

- **Trace** the source of the error as closely as possible
- **Instrument** the code near the error
- **Debug** interactively (*)
- **Explore** question boards

(*) tools may vary across different Rails versions (e.g., `byebug` in Rails v6.1)

Backtracing

TAKE THE TIME TO REALLY READ THE ERROR MESSAGE!

Ruby's error messages are long because you get a **backtrace**: it shows you not only where the method occurred, but the whole chain of currently active (i.e., unreturned yet) function calls that lead to where the error manifested itself.

Some bugs are hard to understand without context. For example:

```
Undefined method `foobar' for nil:NilClass
```

Here, some computation likely failed and returned `nil` instead of the object you expected (but you forgot to check for `nil`). `NilClass` is a special class whose only instance is the constant `nil`.

Obviously, consider using a search engine (and check `development.log`)

Instrument

You can add statements and machinery to your code to understand the issue.

Some possibilities:

- for an object in a view, you can display a detailed description for it:
`<%= debug(@movie) %>` or also `<%= @movie.inspect %>`
- in a controller method, raise an exception to show the value you expect at a specific location: `raise params.inspect`
- in a model or controller, you can log: `Rails.logger.debug("msg")`